Real-time Simulations of Momentum-Conserving Vortices using Affine Velocity Approximations

(Supervisor: Radu Cimpeanu)

Tuesday 12$^{\text{th}}$ May, 2026

Alfie Kunz

Alfie.Kunz@warwick.ac.uk

Submitted to MA395 Essay
Mathematics Institute
University of Warwick

WARWICK
THE UNIVERSITY OF WARWICK

Traditional Eulerian methods of solving and modelling inviscid flows (such as DNS of Navier-Stokes, or approximations via FEM or FVM) often struggle with scalability to high-Reynolds, complex shapes, or simply large areas. Meanwhile Lagrangian methods (such as SPH) struggle with inter-particle interactions in projection, and non-ideal time complexities. We discuss a potential remedy: APIC.

Exciting Possibilities of APIC:

- Suitability for high Reynolds number flows. Spoiler: $\mathrm{Re} = 5420$, for a system of $64,000$ particles running in real-time (~ 120 simulation steps per second).
- Scalability: Well-proven in large-scale tasks, or those with complex boundaries (eg: wind turbines, or Moana!).
- Performance allows for fast convergence to analytical results.
- Versatility for future improvements.

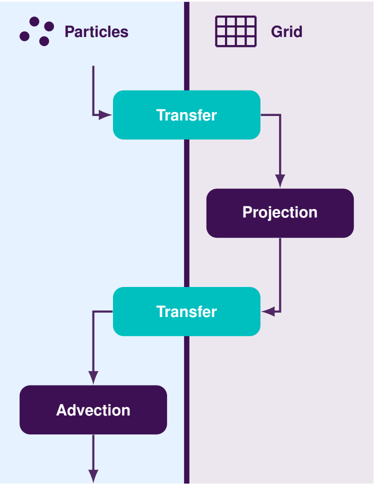
MA395 Submission

Alfie Kunz

Mathematics Institute
University of Warwick

We thus consider hybrid Eulerian-Lagrangian regimes:

- **Transfer Operators:** 'Bilinear Interpolation' to transfer information between particle and grid representations.
- **Grid Projection:** Incompressibility enforcement, by controlling divergence.
- **Particle Advection:** Discrete 'Runge-Kutta' integration for each particle.
- This also allows for trivial classification of fluid structures.



- Using Helmholtz Decomposition and conservation of mass, a velocity field $\underline{u}$ can be written:

$$\boxed{\underline{u}(x,y) = \underline{u}_{df}(x,y) + \underline{u}_{cf}(x,y).}$$

- Taking the divergence, and using scalar potentials to form the pseudo-pressure p, we form the Laplace Equation:

$$\boxed{\nabla \cdot \underline{u} = \Delta p.}$$

- Thus, we can **project** our fluid's velocity field onto its divergent-free basis vector, by computing and then subtracting the pseudo-pressure gradient.
- Note that, via the discretisation of the gradient operator in a FDM regime, it is natural to define a cell's pressure at it's centre, and its two (x,y) velocity components at its right and top edges respectively: a 'Staggered MAC' grid.

Mathematics Institute
University of Warwick

Let s denote the width of each Eulerian cell.

$$(\nabla \cdot \underline{u})(x,y) = \frac{\underline{u}(x+\frac{s}{2},y) - \underline{u}(x-\frac{s}{2},y) + \underline{u}(x,y+\frac{s}{2}) - \underline{u}(x,y-\frac{s}{2})}{s}.$$

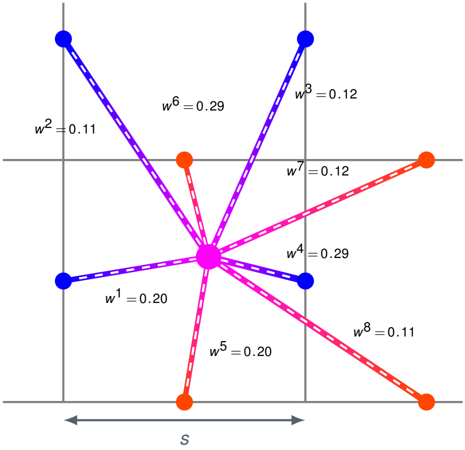
$$(\Delta p)(x,y) = \frac{p(x+s,y) + p(x-s,y) + p(x,y+s) + p(x,y-s) - 4p(x,y)}{s^2}.$$

The PIC Method
Transfer Operators

For each particle p, we form the body of staggered grid nodes, for both x and y velocities, with respective (normalised) weights w_p^i. This allows for the transfer of data (namely velocity) between grid and particles, using 'Bilinear Interpolation'.

$$
\begin{aligned}
w_p^{1,5} &= \frac{\Delta x}{s}\frac{\Delta y}{s}, \\
w_p^{2,6} &= \frac{\Delta x}{s}\left(1 - \frac{\Delta y}{s}\right), \\
w_p^{3,7} &= \left(1 - \frac{\Delta x}{s}\right)\left(1 - \frac{\Delta y}{s}\right), \\
w_p^{4,8} &= \left(1 - \frac{\Delta y}{s}\right)\frac{\Delta y}{s}.
\end{aligned}
$$

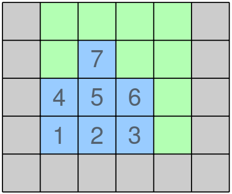
$$
\underline{u}_{\text{grid}} = \frac{\sum_i \sum_p w_p^i \, \underline{u}_p}{\sum_i (m_i := \sum_p w_p^i)}.
$$



Mathematics Institute
University of Warwick

The PIC Method
Projection

- We represent the Laplace equation $\boxed{\nabla \cdot \underline{u} = \Delta p,}$ in our fluid by constructing the matrix equivalent of the negative Laplace operator, called the Coefficient Matrix $\mathbf{A} \in \mathbb{R}^{\text{no. fluid cells}^2}$:

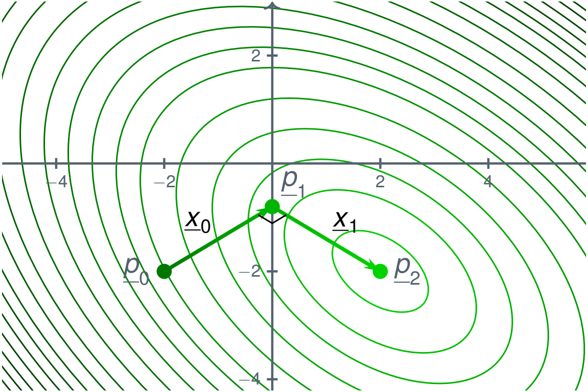


$$\mathbf{A} = \begin{pmatrix} 2 & -1 & 0 & -1 & 0 & 0 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 & 0 \\ 0 & -1 & 2 & 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & 2 & -1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 4 & -1 & -1 \\ 0 & 0 & -1 & 0 & -1 & 2 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$$

- This forms a large linear equation about a sparse, symmetric, positive-definite matrix, $\boxed{\mathbf{A}\underline{p} = -s^2\underline{d},}$ where $\underline{d}$ is the divergence of each cell. This can be solved using the 'Conjugate Gradient' method of gradient descent.

We aim to descend the contours of $f(\underline{p}) = \frac{1}{2}\underline{p} \cdot \mathbf{A}\underline{p} + \underline{p} \cdot (s^2\underline{d})$,

called a quadratic form, by constructing a set of conjugate vectors, and their respective coefficients, that each remove an orthogonal component of vector projection.



This method converges fully within $n_t :=$ (no. fluid cells) iterations.

PIC Framework

Conjugate Gradient Method: The Equations

Instead of a direct solve, we iteratively approximate the solution to $\boxed{\mathbf{A}\underline{p} = -s^2\underline{d},}$, by constructing a basis $\{x_1, ..., x_{n_t}\}$ of $\mathbb{R}^{n_t}$. At each step, k, we compute:

- **The Error:** The left-over divergence unaccounted for:

$$\boxed{\underline{r}_k = -s^2\underline{d} - \mathbf{A}\underline{p}_k.}$$

- **The Conjugate Vector:** 'Gram-Schmidt' orthogonalisation ensures each new direction is conjugate to all the previous:

$$\boxed{\underline{x}_k = \underline{r}_k - \sum_{i<k} \frac{\langle \underline{x}_i, \underline{r}_k \rangle_A}{\langle \underline{x}_i, \underline{x}_i \rangle_A} \underline{x}_i.}$$

- **The Coefficient:** Minimising $f(\underline{p})$ along this new direction, $\underline{x}_k$:

$$\boxed{\alpha_k = \frac{\underline{x}_k \cdot \underline{r}_k}{\langle \underline{x}_k, \underline{x}_k \rangle_A},} \qquad \boxed{\underline{p}_{k+1} = \underline{p}_k + \alpha_k \underline{x}_k.}$$

The PIC Method

Preconditioning, using the '(Modified) Incomplete Cholesky Factorisation with (Drop-Tolerance) Thresholding, Level-Zero (MICT-0)'

This method converges well if $\mathbf{A} \approx \mathbf{I}_{n_t}$, or rather has eigenvalues clumped near 1. Preconditioning transforms the linear system to $\boxed{\mathbf{M}^{-1}\mathbf{A}\underline{p} = -\mathbf{M}^{-1}s^2\underline{d},}$, for preconditioner $\mathbf{M}$.

- If $\mathbf{M} = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is lower-triangular, then we can find the preconditioned error $\underline{z}_k$ by solving the sub-system:

$$\begin{aligned} \underline{r}_k &= \mathbf{L}\underline{a}_k \quad \text{(Forward substitution to solve } \underline{a}_k\text{)}, \\ \underline{a}_k &= \mathbf{L}^T\underline{z}_k \quad \text{(Backward substitution to solve } \underline{z}_k\text{)}. \end{aligned}$$

- **MICT-0 Construction:** We approximate $\mathbf{K} \approx \mathbf{L}$ whilst matching its sparsity, using the concept of 'Drop-Tolerancing':

$$\mathbf{K}_{ij} = \begin{cases} \sqrt{w\mathbf{K}_{ii}} & i = j \\ w & \text{if } j < i,\ |w| \geq \tau \\ 0 & \text{else} \end{cases},$$

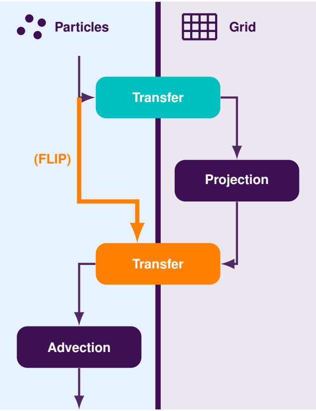
$$w = \frac{\mathbf{A}_{ij} - \sum_{k=1}^{j-1} \mathbf{K}_{ik}\mathbf{K}_{jk}}{\mathbf{K}_{jj}}.$$

Mathematics Institute
University of Warwick

Failures of PIC
FLIP

- **Conservation:** By saving a copy of velocities before grid operations, and transferring the *difference* in velocities back to the particles, we conserve energy in the transfer operations.
- We can blend between PIC and FLIP schemes in the 'Grid to Particle' transfer via the equation:

$$\underline{u}_p = \alpha \cdot \underbrace{\mathrm{BiLerpGP}_{\underline{x}_p}(\underline{u}_{grid}^{new})}_{\text{PIC}} + (1-\alpha) \cdot \underbrace{\left[\underline{u}_p^{old} + \mathrm{BiLerpGP}_{\underline{x}_p}(\underline{u}_{grid}^{new} - \underline{u}_{grid}^{old})\right]}_{\text{FLIP}}$$



Failures of PIC

'So what's really going on here?!' - A Re-consideration of Transfer Operators

- Our grid representation simply does not have enough degrees of freedom, in comparison to the particles! Information is lost during the transfer operators.
- This explains the dampening of PIC, and the instability of FLIP!
- One can show that the loss of angular momentum in the transfers, per particle, is $\boxed{L_p^{\text{loss}} = \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p) \times \underline{u}_g,}$ where $\underline{\chi}_g$ are the grid nodes used in interpolation.
- We must 'upgrade' our interpolation operators with extra energy modes, or rather velocity gradient information, which therefore help with deformation.
- Note that the velocity gradient, $\nabla \underline{u}$, can be decomposed into a symmetric and anti-symmetric matrix, which are responsible for handling shear and rotational deformations respectively.

MA395 Submission
Alfie Kunz

Mathematics Institute
University of Warwick

$$\underline{u}(\underline{x}_g) = \underline{u}(\underline{x}_p) + \mathbf{R_p}(\underline{x}_g - \underline{x}_p), \text{ where } \mathbf{R_p} \in \mathcal{L}(\mathbb{R}^2, \mathbb{R}^2) \text{ is anti-symmetric.}$$

- $\mathbf{R_p} = \omega_p \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \alpha_x \\ \alpha_y \end{pmatrix},$ is thus an angular velocity operator.
- We form the constitutive relation $\underline{L}_p = \mathbf{I}_p \underline{\omega}_p$, where $(\mathbf{I}_p)_{ij} := \sum_k m_k (r_k^2 \delta_{ij} - (x_k)_i (x_k)_j)\, dV, \in \mathbb{R}^{3,3}$, is the particle's <u>inertia tensor</u>. This can be simplified in the rigid case.

$$\omega_p = \frac{L_p^{\text{loss}}}{I_p} = \frac{\sum_g w_p^g \left[(\underline{\chi}_g - \underline{x}_p^{\text{old}}) \times \underline{u}_g \right]}{\sum_g w_p^g ||\underline{\chi}_g - \underline{x}_p^{\text{new}}||^2},$$

- As the name suggests, rigid functions preserve distance locally, so fluid elements cannot shear.

MA395 Submission
Alfie Kunz

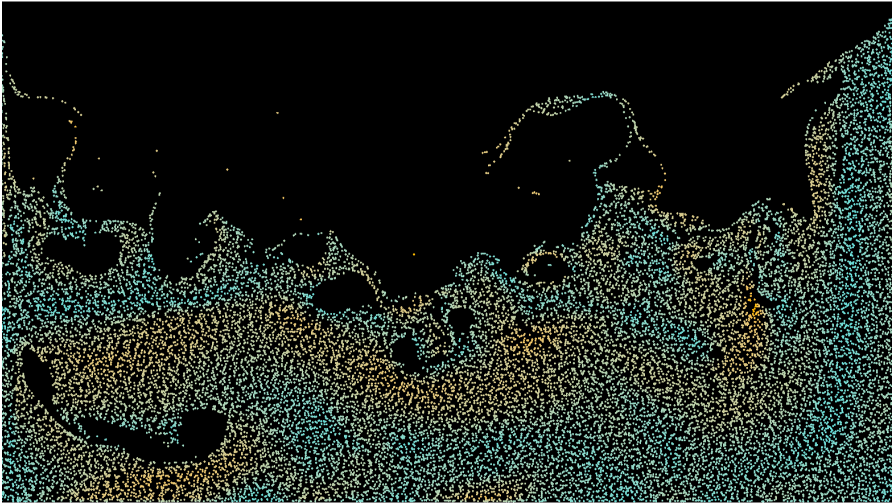
$$\boxed{\underline{u}(\underline{x}_g) = \underline{u}(\underline{x}_p) + \mathbf{C_p}(\underline{x}_g - \underline{x}_p), \text{ for } \mathbf{C_p} \in \mathcal{L}(\mathbb{R}^2, \mathbb{R}^2) \text{ general.}}$$

- We generalise $\mathbf{R_p} \rightsquigarrow \mathbf{C_p}$ as a full velocity gradient matrix: $\boxed{\mathbf{C_p} = \mathfrak{L}_p(\mathfrak{I}_p)^{-1},}$ where $\mathfrak{L}_p$ and $\mathfrak{I}_p$ are generalised angular momentum loss and inertia tensors respectively.
- $\boxed{\mathfrak{L}_p = \sum_g w_p^g \underline{u}_g(\underline{\chi}_g - \underline{x}_p^{\text{old}})^T.}$
- $\boxed{\mathfrak{I}_p = \sum_g w_p^g (\underline{\chi}_g - \underline{x}_p^{\text{new}})(\underline{\chi}_g - \underline{x}_p^{\text{new}})^T.}$ Using the definition of the interpolation operators, we form $\boxed{\underline{\nabla w}_p^i = w_p^i \mathfrak{I}_p^{-1}(\underline{\chi}_i - \underline{x}_p^{\text{old}}).}$
- Note that special care must be taken in the world of staggered MAC velocity components: segment into component faces.

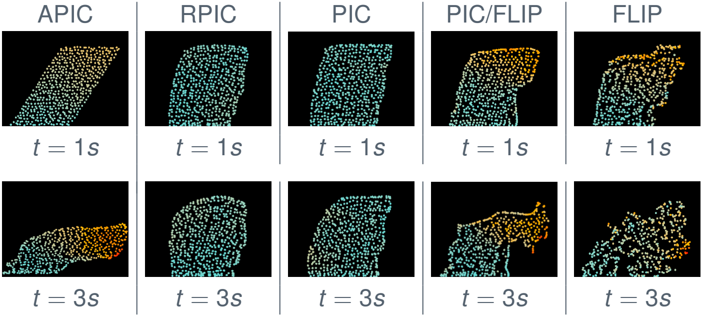
MA395 Submission

Alfie Kunz

Mathematics Institute
University of Warwick

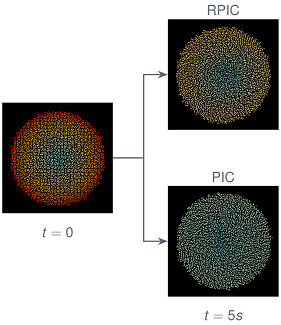


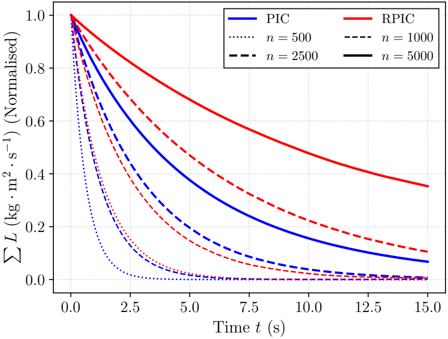
Only schemes that include symmetric components in their velocity gradients (namely FLIP and APIC) can effectively overturn due to shear deformations. Vortices emerge only in these regimes.



Results
Rotation of Fluid Elements

Only schemes that include anti-symmetric components in their velocity gradients (namely FLIP, RPIC, and APIC) can effectively handle rotational deformations. RPIC both conserves angular momentum better than PIC, and converges much faster to the linear decay limit.





From this, we see that APIC is the only stable scheme that can both produce and sustain vortex motion.

Mathematics Institute
University of Warwick

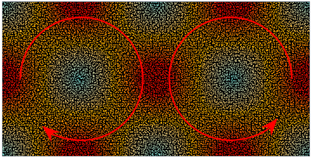
Results

APIC Analytical Case Study: Taylor-Green Vortex

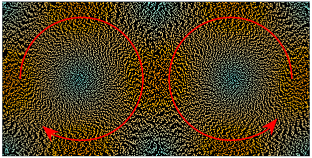
Comparing against the analytical (2x2) Taylor-Green vortex shows remarkable convergence to the true inviscid flow equations.

$$\underline{u}(x, y, t) = \begin{pmatrix} \sin(\kappa_x x)\cos(\kappa_y y) \\ -\cos(\kappa_x x)\sin(\kappa_y y) \end{pmatrix} e^{-2\nu|\underline{\kappa}|^2 t},$$

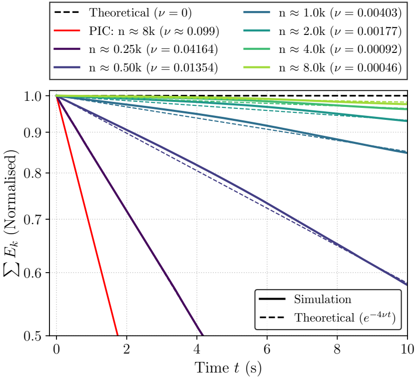
$$E_\kappa = E_0 e^{-4\nu\kappa^2 t}.$$



$t = 0$



$t = 10.00s$ (APIC)



Conclusion

Results, Reflections, and Future Work

MA395 Submission

Alfie Kunz

Result: Taylor Green Vortex Reynolds Number

We derive an upper bound result of **Re = 5419.69** (16,000 APIC particles per vortex) - a remarkable value for real-time CFD.

- For reference, standard PIC yields Re = 10.27 in the same setup.
- Whilst RPIC effectively resolves rotational deformations, its lack of shearing potential makes it ill-equipped for most real-world applications: we need the inclusion of symmetric velocity gradient components, as in APIC.
- FLIP shows promise, if not for instability and lack of ‘physics’.
- Whilst great, APIC is not perfect! Particle clumping under large forces, a lack of physical relation to surface tension or viscosity applications, a lack of persistent bubbles, to name but a few.
- Promising research has already given insights into improving these behaviours! Eg: APIC/FLIP hybrids, or PolyPIC.

Thank you for your time!
Any Questions?